

模组介绍

本模组是一个工具兼模板。

本模组的功能是允许开发者模拟原版决议的形式，注册可以在任意GUI界面中显示的决议。在使用时需要将本文件中的所有部分，除了

`common\scripted_triggers\scripted_decision_test_triggers.txt`
`,common\scripted_effects\scripted_decision_test_effects.txt,interface\scripted_decision_template.gui,common\scripted_guis\scripted_decision_template.txt` 四个文件之外全部复制到需要装载该功能的模组中。

决议与决议组的注册与参数

`create_scripted_decision` 与 `create_scripted_decision_category` 效果可以创建一个空决议或决议组。决议组以 `character` 的形式储存在游戏代码中，在使用该效果之后，一个变量 `new_scripted_decision/new_scripted_decision_category` 将会可用，允许开发者用变量的形式访问和储存这两个角色，建议将该角色用一个开发者自己定义的变量储存。

开发者可以通过 `var:my_variable = {}` 的方式访问该角色的作用域。

决议的参数

在决议角色作用域中，开发者可以用以下形式定义如下参数：

- `set_character_name = " MyDecisionTokenKey"` 这相当于决议的注册名。决议的注册名决定了访问决议的本地化名、介绍、各个 `Trigger` 和 `Effect` 所调用的标识符值。需要注意的是，决议的注册名键值不能直接接在 `set_character_name =` 后，必须用引号包裹，且在 **引号内** 的前面（即第一个引号后）必须加一个空格，否则将会把标识符键值对应的本地化直接设置为决议的注册名，导致无法正确读取其 `Trigger` 和 `Effect` 等。如下标记了通过注册名读取的数据：
 - `remove_effect, remove_trigger, cancel_effect, cancel_trigger, available, activation, timeout_effect, complete_effect` :与原版完全相同。以上文为例，注册名为 `MyDecisionTokenKey` 的决议的“移除时效果”为 `MyDecisionTokenKey_remove_effect`。需要注意的是并没有 `allowed`，且每个决议只服务于一个国家，需要为这个国家单独注册。其默认情况也与原版完全相同：如果不注册 `available`，那么作为决议或者 `selectable_mission = yes` 的任务将会被视为始终为真，但是作为一般的任务将会被视为始终为假，不会自动完成：因此你可以不必为每一个都注册。这些 `trigger` 和 `effect` 分别需要注册在 `scripted_triggers/` 和 `scripted_effects/` 文件夹中的任意文件中。
 - `desc` 由于代码层面上的问题，`desc` 必须注册，否则会直接显示本地化键。
 - `custom_cost_trigger` 与其他 `trigger` 的注册方式相同。和原版类似，但是不同的是它与 `cost` 是可以共存的。
- `var:icon = {}` 这可以访问决议的图标，`icon` 也是一个用变量储存在决议中的 `character` 数据类型，通过名称储存 `GFX` 数据。
 - `set_character_name = GFX_XXX` 在 `icon` 中注册，但是必须保证该 `GFX` 键值不是一个本地化键。
- `var:custom_cost_text = {}` 与原版的 `custom_cost_text` 键功能相似，这用于储存自定义的花费本地化键。这会覆盖默认的使用政治点数的本地化键，但是政治点数的 `cost` 仍然生效。
 - `set_character_name = " loc_key"` 在 `custom_cost_text` 中注册，`loc_key` 可以替换为自定义的本地化键。`loc_key, loc_key_block` 和 `loc_key_tooltip` 的功能与原版相同，如果对 `custom_cost_text` 使用了 `set_character_name`，那么必须对这三项都进行注册。

- `cost` 一个变量，决议的政治点数花费，通过 `set_variable` 定义在决议中。
- `days_mission_timeout` 一个变量，任务截止前的默认时间，通过 `set_variable` 定义在决议中。设置为非零值将使决议被视为一个任务。
- `selectable_mission = yes` 一个效果，将任务设置为需要手动点击才能完成。在决议中调用该效果后生效。
- `is_good = yes` 同原版效果
- `days_remove` 一个变量，决议或者任务默认持续的时间，通过 `set_variable` 定义在决议中。设置为非零值将使决议或者任务在完成之后需要时间移除，否则将会立即移除。
- `days_re_enable` 一个变量，任务默认进行的时间，通过 `set_variable` 定义在决议中。设置为非零值将使决议或者任务在移除或者取消之后需要时间才能重新出现。注意：即使设置为 0，仍然需要至少一天的重置时间。通过在效果块中调用 `re_enable_scripted_decision` 效果可以直接绕过刷新时间。
- `fire_only_once = yes` 同原版，使得决议的刷新计时器不再运行。但是可以被 `re_enable_scripted_decision = yes` 绕过。
- `priority` 一个变量，决定决议在决议组中的显示顺序优先级，优先级相同的决议根据决议状态和加入决议组的顺序显示。
- `cancel_if_not_visible = yes` 使得决议在不可见时自动取消。
- `visible_while_not_activated = yes` 本模组新增的参数，这允许使得任务得以在没有激活的情况下仍然可见。

决议组的参数

在决议组角色作用域中，开发者可以通过以下形式定义如下参数：

- `set_character_name` 这点同决议，略。通过注册名读取的数据，注意，决议组的定义中不包含 `desc`：
 - `visible, available` 这点同原版。
- `visible_when_empty = yes` 同原版
- `icon` 同决议
- `priority` 一个变量，决定决议在决议组中的显示顺序优先级，优先级相同的决议根据决议状态和加入数组的顺序显示。
- `add_to_category` 通过如下方式将决议加入决议组：

```
var:my_decision_category = {
    var:PREV.my_decision = {
        add_to_category = yes
    }
}
```

其中 `PREV` 代表引用前一级别的变量。如果是临时变量或者变量直接定义在决议组中则不需要。

在GUI中的注册

想将自定义决议组列加入 GUI 中，首先需要打开本模块的

`\interface\scripted_decision_template.gui` 文件，将

`scripted_decision_container_category_entry` 和 `scripted_decision_container_entry` 两个窗口块整个复制到你需要插入决议组的 GUI 文件中，注意在复制过去之后这两者仍然必须作为一个独立的窗口。随后将 `scripted_decision_container` 窗口插入你需要显示决议组列的窗口中。

随后，打开 `common\scripted_guis\scripted_decision_template.txt` 文件，将其中的 `triggers`, `effects`, `dynamic_lists`, `properties`（注意不包含其他部分）中的全部内容复制到你想要显示决议组列的 GUI 的 `scripted_gui` 文件的对应部分中。

接下来需要确定在决议组列中出现的决议组：找到 `triggers` 中的

`scripted_decision_gui_calculator_visible`，在其中的 `mtth` 一行之前将你所需要的决议组全部放入临时数组 `scripted_all_decision_category_temp_list` 中即可。

除此之外，在这里需要特别注意的是，即使决议组没有不出现在 GUI 中，正在进行中的决议与任务仍然可以正常运作（但是诸如可见性、中止条件等因素将仍然起作用），开发者可以放心。

显示控制

`scripted_decision_generate_decision_category_array` 和

`scripted_decision_generate_decision_array` 这两个 `trigger` 是用于生成当前在 `gui` 中可见的决议的。一般来说，这两个 `trigger` 已经包含在了本模块提供的模板外观控制 `mtth` 中。

模板外观控制 `mtth` 在本模块中已经提供，且对于绝大多数决议窗口，都可以直接调用这几个模板以实现外观控制。而对于其他信息，已均列在下方

除此之外，本模块适配了滑动条：只需要将 `gui` 文件前的 200 多条空窗口复制进你所需要显示决议 `gui` 的文件中即可。其中，`gui` 文件中的 `scripted_decision_slider_controller_grid` 是用于控制高度的元件，在 `scripted_gui` 文件中需要按照如下注册：

```
scripted_decision_slider_controller_grid = {
    array = scr_dec_decision_slider_grid_array
    value = scripted_decision_slider_grid_entry
    index = scripted_decision_slider_grid_index
    change_scope = no
    entry_container =
"scripted_decision_container_slider_controller_container_[]?
scripted_decision_slider_grid_entry]"
}
```

其中 `scr_dec_decision_slider_grid_array` 是在中计算出来的高度数组，具体来说，将高度计算出来之后，将按照 100 拆分，但是最后一个需要在 100 到 200 之间。这样就实现了控制滑槽判定。

GUI内置参数

在 `Triggers` 中的计算器部分定义了许多临时变量作为参数。这意味着开发者可以调用它们。

- `scr_det_[cat]_{element}_{x/y}_modifier` 诸如这种格式的均为决议或元件的位置调整法则，在 `trigger` 中定义，在 `mtth` 中调用来调整位置。
- `scr_dec_cost_and_timer_text_selectable_{x/y}_modifier` 与其他的不同，`cost_and_timer_text` 的位置会根据决议是否是一个需要选择的决议来调整位置。`scripted_decision_selectable_trigger` 这一条件可以用于判断决议是否是一个需要选择的决议。

- `scr_dec_decision_height_sum`, `scr_dec_decision_category_height_sum`, `scr_dec_decision_gui_height` 这三者通常用于累加高度，它们在完整运行一次后的最终值应当是决议块的实际高度，具体实现参见本模组的模板 `mtth`。
- `scr_dec_decision_entry_[x/y]_base` 这允许决定单个决议相对于决议组中的决议列表的位置，默认情况下 `y` 位置为 `scr_dec_decision_height_sum`，这是一个对 `scr_dec_decision_height` 参数累加得到的值。
- `scr_dec_decision_height` 决定了单个决议占用的的高度，默认情况下为 `scr_dec_grid_height`，这用于累加 `scr_dec_decision_height_sum` 来决定决议的位置。
- `scr_dec_grid_height` 决议组中的参数，决定了单个 `grid` 的默认高度。
- `scr_dec_category_[x/y]_base` 这允许决定单个决议组相对于 GUI 中的决议组列表的位置，默认情况下 `y` 位置为 `scr_dec_decision_category_height_sum`，这是一个对 `scr_dec_decision_height` 参数累加得到的值。
- `scr_dec_decision_category_height` 这决定了单个决议组的高度，默认情况下为其中决议的数量乘以单个 `grid` 的高度，加上决议列表的 `y` 位置 `scr_dec_decision_grid_y_base`。
- `scr_dec_decision_grid_[x/y]_base` 这决定了决议列表相对于决议组的位置。
- `scripted_decision_category_entry`, `scripted_decision_entry` 当前正在显示的决议组或者决议条目。
- `scripted_decision_[category]_gui_should_show = yes` 一个条件，判断决议（组）是否应该展示在 `gui` 中（与可见不同，决议组可以收起，此时决议不展示但是决议仍然可见）
- `scripted_decision_tooltip_loc_key` 一个本地化键，是默认的决议提示。

对GUI有限的修改

开发者可以在其中增加或修改一部分 GUI 原件，以下给出了一个简单的例子：

例如，本模组并没有实现**决议组**的 `desc`，`picture` 和决议组内置 `gui` 功能，因为这需要一个自动调整大小的文字背景框和动态高度计算。

因此，开发者可以将决议组内置 `gui`、`desc` 或 `picture` 作为新 `gui` 元件写入 `scripted_decision_container_category_entry`，并且利用 GUI 内置参数，模仿 `mtth` 文件中的写法来调整位置。随后对每一个元件在 `triggers` 中定义 `visible`，并且对 `scripted_decision_category_entry` 变量进行 `check_variable` 来判断其可见性，或通过 `properties` 和 `scripted_localisation` 等来调整其图像或文字。并在内置参数中调整 `scr_dec_decision_category_height` 的数值。

类似的，这也可以用于实现在单个决议之后附加这单个决议的独有 `gui`，或者单独调整某个决议或决议组的外观。

除此之外，为了防止重名，可以三个窗口的名称，但是也需要同步更改 `scripted_gui` 文件中的 `dynamic_list` 中调用的窗口名称。

以及除了两个 `icon` 外，其他图像元素都可以修改，但是需要注意保留原版的图像的格式、帧数和 `FX` 效果，并且需要仿照注册 `mtth` 文件和 `properties`。

可调用的参数和可执行的效果

决议

变量

- `days_mission_timeout_left`, `days_remove_left`, `days_re_enable_left` 均为变量，在决议范围内调用，且可以像一般变量一样修改。尤其需要注意：必须是正在进行的任务、正在移除的决议或正在重置的决议或任务才能通过修改来影响。

效果

- `cancel_scripted_decision` 在决议作用域内调用。可以直接取消一个决议，并且调用 `cancel_effect`
- `cancel_scripted_decision` 在决议作用域内调用。可以直接取消一个决议，不调用 `cancel_effect`
- `complete_scripted_decision` 立刻完成一决议或任务，并调用 `complete_effect`，绕过 `cost` 和 `available` 的限制。
- `scripted_decision_start_remove` 立刻开始移除一个决议，不调用 `complete_effect`，绕过 `cost` 和 `available` 的限制。
- `remove_scripted_decision` 立刻移除一个正在移除中的决议，并调用 `remove_effect`。
- `scripted_decision_end_remove` 立刻移除一个正在移除中的决议，不调用 `remove_effect`。
- `timeout_scripted_mission` 立刻使得当前决议时间到期，并调用 `timeout_effect`
- `activate_scripted_mission` 立刻启用一个任务，绕过 `activation` 和 `visible`，但是注意如果同时具有 `cancel_if_not_visible` 和 `visible_while_not_activated` 会在任务启用的同时立刻取消。
- `re_enable_scripted_decision` 立刻使得当前决议重置。

条件

- `scripted_decision_should_showing` 这用于判断决议当前是否实际可见（即在 `gui` 中的可见情况，但是即使不在 `gui` 中注册或者被收起了也能正常使用）。
- `scripted_decision_can_be_complete` 判断当前决议是否可以通过效果完成，不考虑 `cost` 和 `available` 的限制。内部包含了 `scripted_decision_should_showing`
- `scripted_decision_available_trigger` 判断当前决议的 `available`（含决议组的 `available`）和 `cost`
- `scripted_decision_cost_trigger` 判断当前决议的 `cost`
- `scripted_decision_available_is_true` 判断当前决议的 `available`（含决议组的 `available`）
- `scripted_decision_is_mission` 判断当前决议是否为一个任务
- `scripted_decision_is_visible` 判断当前决议的 `visible`（含决议组的 `visible`）
- `scripted_mission_can_be_auto_complete` 判断当前决议是否可以被自动完成。

决议组

暂无

其他有关事项

如何制作效果提示？

由于一些不可抗力原因，暂时无法直接调用决议的本地化名字（如果调用本模组内置的 `scripted_localisation`，名字后面将会有有一个强制换行）。可以改用以下方法注册 `tooltip`，其他同理。

```
meta_effect = {  
    text = {  
        custom_effect_tooltip = {  
            localisation = COMPLETE_DECISION_LOC  
            DECISION_NAME = [t_decision]  
        }  
    }  
    t_decision = "[?THIS.My_Decision.GetName]"  
}
```

```
COMPLETE_DECISION_LOC:0 "完成决议$DECISION_NAME$"
```